

85/100

Técnicas de Diseño de Algoritmos
Primer Cuatrimestre 2025
1er Parcial

Nombre y Apellido	L.U.	N°Orden
-------------------	------	---------

Turno: ☒ Mañana ☐ Tarde/Noche | Materia: ☐ TDA ☒ AED 3

Duración: 5 horas. Este examen es a libro cerrado e individual. La nota de aprobación es de 60/100 puntos.

1. Preguntas de opción multiple

Cada pregunta tiene 4 opciones y una sola opción correcta. Las respuestas correctas (Bien) suman 5 puntos. Las respuestas incorrectas (Mal) restan 2 puntos. Las preguntas no respondidas no se consideran como incorrectas y suman 0 puntos. Marcar en el cuadrado la opción que consideren correcta.

Pregunta 1 Sea la siguiente función recursiva que calcula el número combinatorio $\binom{N}{K}$

$$\binom{N}{K} = \begin{cases} 1, & \text{si } k = 0 \vee k = n \\ 0, & \text{si } k < 0 \vee k > n \\ \binom{N-1}{K-1} + \binom{N-1}{K}, & \text{si no} \end{cases}$$

¿Cuál es la complejidad temporal de calcular el número $\binom{N}{K}$ utilizando Programación Dinámica?

☐ $\Theta(N^2)$

☐ $\Theta(K^2)$

☒ $\Theta(N * K)$ ✓

☐ $\Theta(N^2 * K)$

Pregunta 2 Dado un grafo con N nodos y M aristas. ¿Cuál es la complejidad espacial de la representación como matriz de adyacencia?

☒ $\Theta(N^2)$ ✓

☐ $\Theta(N + M)$

☐ $\Theta(N * M)$

☐ $\Theta(M)$

Pregunta 3 Si en la representación de un grafo de N nodos (1 a N) y M aristas como lista de adyacencia, reemplazo los vectores que contienen a los vecinos de cada nodo por tablas hash (*) (ej: unordered_set en C++, HashSet en Rust, dict en Python), podré:

☐ Poder recorrer los vecinos de un nodo v dado en orden ascendente en $O(d(v))$

☐ Tener complejidad espacial $O(N)$

☒ Saber si dos nodos u y v son vecinos en $\Theta(1)$ ✓

☐ Saber si dos nodos u y v tienen vecinos en común en $O(\log(N))$

(*) Durante el parcial no se responderán consultas sobre lo que es una tabla hash.

Pregunta 4 Dado el dígrafo con 5 nodos y aristas: (1, 2), (1, 3), (2, 4), (3, 4), (4, 5). ¿Cuál de los siguientes es un recorrido de DFS válido?

☒ 1 - 2 - 4 - 5 - 3 ✓

☐ 1 - 2 - 4 - 3 - 5

☐ 1 - 2 - 3 - 4 - 5

☐ 1 - 3 - 2 - 4 - 5

Pregunta 5 Dado un grafo dirigido con nodos del 1 al N , donde el nodo i ($1 \leq i \leq N$) tiene asociado un string s_i (todos los s_i son no vacíos y distintos) en el cual hay una arista (i, j) si y solo si s_j es un sufixo de s_i . ¿Cuál de las siguientes afirmaciones es falsa?

- ☐ De un nodo i pueden salir a lo sumo $|s_i|$ aristas
- ☐ El grafo tiene bucles (pares (i, i))
- ☐ El grafo es transitivo
- ☒ El grafo puede tener ciclos dirigidos simples con más de un nodo ✓

(*) $|s|$ es la cantidad de caracteres de s

Pregunta 6 Si represento a una función recursiva f (que al computarla no genera una recursión infinita) como un grafo dirigido donde cada uno de los N nodos es un estado y una arista (a, b) significa "La función se llama a sí misma con el estado b para procesar el estado a ". ¿Cuál de las siguientes afirmaciones es correcta?

- ☐ Puede haber ciclos en el grafo
- ☐ Siempre podemos calcular la función f para todos los estados en $O(N)$
- ☐ Los casos base tienen grado de entrada 0
- ☒ Llamar a la función con el estado x generará tantas llamadas al estado y como caminos hay de x a y en el grafo ✓

Pregunta 7 La relación entre los nodos de un grafo no dirigido definida como $R(a, b)$ si y sólo si a y b pertenecen a la misma componente conexa, es:

- ☒ Una relación de equivalencia ✓
- ☐ Una relación de orden (parcial o total)
- ☐ Anti-simétrica ($R(a, b) \implies \neg R(b, a)$)
- ☐ Una relación irreflexiva (no existe $R(a, a)$)

Pregunta 8 La complejidad espacial de la memorización de una DP con N estados y M transiciones entre estados, implementada de forma top-down recursiva, es:

- ☐ $\Theta(M)$
- ☐ $\Theta(N + M)$
- ☒ $\Theta(N)$ ✓
- ☐ $\Theta(N * M)$

2. Preguntas abiertas de respuesta corta

Respuesta Bien: +5 puntos. Respuesta Regular, Mal o sin completar: +0 puntos.

Pregunta 9 En el algoritmo de DP de Floyd Warshall visto en clase, ¿Qué se calcula en cada estado de la DP?

BR PARA CADA PAR DE VERTICES (i, j) SE CALCULA LA MINIMA DISTANCIA ENTRE EL VALOR MEMORIZADO Y ENTRE AGREGAR EL VERTICE k AL CAMINO CON $\min(d_{ij}^{k-1}, d_{ik}^{k-1} + d_{kj}^{k-1})$ ✓
EXPLICAS COMO SE CALCULA, NO QUÉ SE CALCULA.

Pregunta 10 En el algoritmo de DP de Bellman Ford visto en clase. ¿Qué se calcula en cada estado de la DP?

R DE LA MISMA MANERA QUE ARRAIA, CON FLOYD, PERO EXTENDIENDO EL CAMINO DE LARGO $k-1$ A k AGREGANDO UNA ARISTA AL CAMINO Y COMPARANDO CON EL VALOR MEMORIZADO X
CADA ESTADO CALCULA EL CAMINO MINIMO DESDE EL ORIGEN HASTA i CON k ARISTAS NO ES DE LA MISMA MANERA QUE CON FLOYD

Pregunta 11 Sea G un grafo dirigido (dígrafo) donde los nodos son los números de 1 a N y existe una arista (a, b) si y sólo si existe un número primo p tal que $b = a * p$. ¿Qué significa que exista un camino de u a v en G ?

B SIGNIFICA QUE N ES MULTIPLO DE u , SIENDO EL CAMINO MÁS LARGO A v EL QUE CONTIENE A TODOS SUS MULTIPLOS, EN PARTICULAR CONTIENE A SU FACTORIZACIÓN EN PRIMOS. X
CONSIDERO QUE, A PESAR DE QUE LA SEGUNDA PARTE ESTÁ MAL, ENTENDISTE NO NECESARIAMENTE

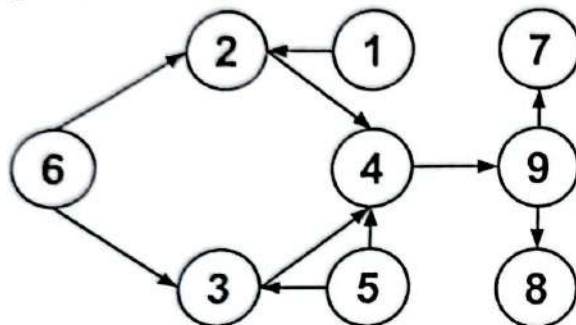
Técnicas de Diseño de Algoritmos
Primer Cuatrimestre 2025
1er Parcial

Nombre y Apellido	L.U.	N°Orden
-------------------	------	---------

Turno: ☒ Mañana ☐ Tarde/Noche Materia: ☐ TDA ☒ AED 3

Duración: 5 horas. Este examen es a libro cerrado e individual. La nota de aprobación es de 60/100 puntos.

Pregunta 12 Indique un orden topológico válido del siguiente grafo dirigido acíclico:



6-3-4-9-7-8-2-1-5 X

3. Ejercicios a Desarrollar

Ejercicio Bien: +10 puntos. Ejercicio Regular: +5 puntos. Ejercicio Mal o sin completar: +0 puntos.

Pregunta 13 Dado un grafo dirigido $G = (V, E)$ con nodos números de 1 a N . Se tiene asociado a cada nodo v un peso p_v entero positivo.

Se pide diseñar un algoritmo o utilizar uno existente que en $O(N^3)$ (espacial y temporal) determine cada par ordenado de nodos (u, v) si es posible llegar de u a v en el dígrafo dado y, si es posible, determinar el menor peso P tal que es posible llegar utilizando como nodos intermedios (sin contar u ni v) solo a nodos con un peso menor o igual a P .

En caso de utilizar una modificación a un algoritmo existente, solo es necesario detallar cuales serían estas modificaciones.

Pregunta 14 Dado un cubo de Rubik, definimos el estado del Cubo como una posición definida por el color de cada celda que hay en cada cara (cada cara tiene un identificador que permite diferenciarlas). El cubo se considera resuelto si cada cara tiene únicamente celdas de un mismo color. Una jugada válida consiste en rotar una cara del cubo 90 grados en sentido horario o anti-horario.

Se pide modelar el cubo con un grafo donde:

- Cada nodo represente un estado del cubo.
- La distancia entre dos nodos u y v sea igual a la cantidad de jugadas necesarias para convertir el estado representado por u en el estado representado por v .

Pregunta 15 Sea el problema Sudoku N el problema de rellenar una matriz de $N \times N$ con números de 1 a N tal que no haya dos celdas con el mismo número en la misma fila ni en la misma columna. Sean estas dos soluciones con Backtracking a dicho problema:

```

1 def Sudoku(estado: list[list[int]], celda: int) -> int:
2     if celda == N * N:
3         return int(es_valido(estado))
4     fil, col = celda // N, celda % N
5     retorno = 0
6     for num in range(1, N+1):
7         estado[fil][col] = num
8         retorno += Sudoku(estado, celda + 1)
9         estado[fil][col] = 0
10    return retorno

```

```

1 def Sudoku_poda(estado: list[list[int]], celda: int) -> int:
2     if celda == N * N:
3         return 1
4     fil, col = celda // N, celda % N
5     retorno = 0
6     for num in range(1, N+1):
7         estado[fil][col] = num
8         if es_valido(estado):
9             retorno += Sudoku_poda(estado, celda + 1)
10        estado[fil][col] = 0
11    return retorno

```

Demostrar que:

- A) La cantidad de estados visitados por Sudoku es $\Theta(N^{(N^2)})$
- B) La cantidad de estados visitados por Sudoku_poda es $O((N!)^N \cdot N^2)$

Puede usar lo siguiente sin necesidad de demostrarlo:

$$\sum_{i=1}^{i=n} p^i \in \Theta(p^n)$$

Cada una de las 2 demostraciones puede estar Bien, Regular o Mal. Para tener el ejercicio Bien ambas deben estar Bien. Para tener el ejercicio Regular se necesita que ambas estén Regular o al menos una esté Bien. Caso contrario, se tendrá el ejercicio Mal.

Pregunta 16 Sea el problema:

Dado un conjunto S de N items (p_i, v_i) y un peso límite P , no negativo. Determinar el máximo valor posible V tal que existe un subconjunto S' de S tal que:

- A) La suma de los p_i con i en S' es menor o igual a P .
- B) La suma de los v_i con i en S' es igual a V .

Dicho de otra forma, el máximo valor total entre los subconjuntos cuyo peso total no excede P .

Y dada la siguiente solución (S es dado como una lista):

```

1 def Mochila(P: int, S: list[tuple[int,int]]) -> int:
2     if P < 0: return -inf
3     if len(S) == 0: return 0
4     (peso, valor) = S[0]
5     return max(Mochila(P, S[1:]), Mochila(P-peso, S[1:])+valor)

```

Demostrar que $Mochila(P, S)$ (con los P y S del enunciado original) devuelve la respuesta al problema planteado, independientemente del orden de los elementos en S .

13) Modifico el algoritmo de Floyd que ya encuentra si hay un camino entre u y v para cada par de nodos en $O(n^3)$ Temporal y $O(n^2)$ espacial para que haga lo siguiente:

Guardo una matriz MINP de $N \times N$ donde para cada (i, j) en cada iteración k :

- si no hay camino con el nodo k no hace nada
- si hay camino:

- guarda en $\text{MINP}[i][j] =$

- el mínimo entre el valor que ya tenía entre i, j y el máximo entre $(\text{MINP}[i][k], P_k, \text{MINP}[k][j])$

(la matriz MINP se inicializa con todos ∞)

asi tome el minimo peso de los caminos que encuentre de i a j teniendo en cuenta que un camino que va de i a k a j tiene peso maximo el maximo entre los nodos de i a k , el peso de k , y los nodos de k a j . agregar la matriz es $O(n^2)$ al inicializarla temporalmente ($O(n^2) \in O(n^3)$) y espacialmente tambien. luego el codigo son comparaciones e ifs que son $O(1)$ por lo que mantiene la misma complejidad que floyd.

luego de ejecutar este algoritmo $\text{MINP}[i][j]$ tiene valor $< \infty$ si hay camino y este será el menor peso mayor al resto del camino, y menor entre todos los caminos de i a j .

14) define el grafo como:

los vertices son los posibles estados del cubo, donde el vertice E_i ~~tiene~~ (para cualquier i , entre la cantidad de estados) tiene 12 aristas que salen, dirigidas, dos para cada cara, una para girarla en sentido horario y otra en el sentido anti-horario, cada una dirigida al estado E_j al cual terminaria el estado E_i si se le aplica ese movimiento.

Como cada arista representa un movimiento válido, si partimos del estado e y contamos la distancia que recorremos, al hacer cualquier camino a un estado v esa será por definición la cantidad de movimientos necesarios ~~el camino sea la lista de movimientos~~ al ser todos ellos válidos.

LA CANTIDAD DE
MOVIMIENTOS NECESARIOS
PARA EL
MINIMO.

✓
QUEDA UN POCO MÁS DIRECTO EL MODELAJO CON
UN GRAFO NO DIRIGIDO PERO ESTÁ BIEN.

15) A)

Un estado se define como una posible solución, al no tener
pedas nomos a permuta ^{repetir} número en la misma fila
o columna, por lo que para cada casillero hay N opciones,
hay N^2 casilleros por lo que para cualquier N dado
nomos a formar las N^2 posibles configuraciones. ✓

B) ahora si para poner un número tenemos solo en cuenta
los que son válidos y rellenamos secuencialmente
la primera fila, nomos a tener que en el primer casillero
tenemos N opciones, en el segundo $N-1$, ... hasta 1 sola opción,
por lo que para esa fila son $m!$ posibilidades. ✓

la segunda fila empieza con $(m-1)$ opciones ya que está tomada
la casilla de arriba por lo que la segunda fila tiene $(m-1)!$
opciones, así llegamos a la última con $1!$, juntando esto
tenemos:

$$m! \cdot (m-1)! \cdot (m-2)! \cdot (m-3)! \cdot \dots \cdot 2! \cdot 1! = \prod_{i=0}^{i=m-1} (m-i)!$$

~~que es menor a que si a cada término lo~~

si a cada término con 1 lo reemplazamos por $m!$ tenemos $(m!)^m$
como cota superior. ✓

pero como "elegimos" darle N opciones a la primer casilla tenemos
que multiplicar por N^2 para ~~las otras casillas~~ ^{posibilidades}
considerar empezar en cualquier casillero. aunque no se use
el algoritmo en si, los estados visitados se forma de la
misma manera.

entonces:

$$O\left(\prod_{i=0}^{i=m-1} (m-i)! \times N^2\right) \in O\left((m!)^m \times N^2\right) \quad (m=N)$$

PIE ESTADOS VISITADOS $\rightarrow$ EL N^2 ES POR LA
CANTIDAD DE INDICES POSIBLES

16) lo pruebo por inducción en el tamaño de S :

- si S tiene tamaño 0 entonces para cualquier peso P el máximo valor tiene que ser 0 ya que está vacío, y $MOCHILA(P, \{\})$ en $len(S) = 0$: RET 0 mas da 0 por lo que se cumple el caso base.
- ahora si S tiene un tamaño cualquiera ≥ 1 y lo podemos representar como $[x:S]$ (casi Haskell) donde x es el primer elemento, asumo como hipótesis inductiva que

$MOCHILA(P, S)$ y $MOCHILA(P - PESO_x, S)$ dan los máximos valores posibles correspondientes a V_p y V_{p-x} .

→ NO ESTÁ MUY BIEN DEFINIDA LA HI

ahora $MOCHILA(P, [x:S])$ se define por la función por: $\max(MOCHILA(P, S), MOCHILA(P - PESO_x, S) + VALOR_x)$

y por HI podemos saber que:

$$MOCHILA(P, [x:S]) = \max(V_p, V_{p-x} + VALOR_x)$$

asi tendríamos el máximo valor de poner x o no al subconjunto que ya maximizaba el valor, obteniendo el máximo valor posible de los $[x:S]$ elementos a un peso $\leq P$.

Aclarar que si agregas x excede el peso V_{p-x} sería $-\infty$ por HI y que en ningún momento se asume algo del orden de los elementos, solo que $|S| \geq 1$, habiéndolo cubierto $|S| = 0$.